

Definition

A context-free grammar is a 4-tuple (V, Σ, R, S) , where

1. V is the set of variables
2. Σ is a finite set, called terminals.
3. R is a set of Rules, with each rule being a variable and a string of variables & terminals
4. S is the starting variable.

☆ context free grammar is a collection of substitution rules also known as productions.

☆ Each rule appears as a line in the grammar.

☆ Each line comprises of a symbol, arrow, and a string.

Ex: 

☆ The string can consist of variables, and other symbols called terminals.

☆ Variables are represented using capital letters, terminals are represented using lowercase letters, numbers or special symbols.

☆ One variable is designated as the starting variable.

First we will look at the pumping lemma.
Pumping lemma for CFLs.

If A is a context free language, then there is a number p (the pumping length) where, If s is a string in A of length at least p , then s may be divided into five pieces $s = uvxyz$, satisfying the conditions

1. $uv^ixy^iz \in A, \forall i \geq 0$
2. $|v| > 0$ ← either v or y has to be a non-empty string
3. $|vxy| \leq p$

Chomsky Normal Form (CNF)

When working with CFGs, it is often convenient to have them in simplified form.

This useful, when we have algorithms working on CFGs.

Definition

Chomsky Normal Form (CNF)

context free grammar is in CNF if every rule is in the form of:

$$A \rightarrow BC$$

$$A \rightarrow a$$

a is a terminal, and A, B, C are any variable except that B & C may not be start variable.

we also allow $S \rightarrow \epsilon$ for the start variable (only for the start variable)

Ambiguity: Sometimes grammar can generate the same string in several different ways.

If a grammar generates a string in several different ways, we say that string is derived ambiguously in that grammar.

If grammar generates some string ambiguously, we say that the grammar is ambiguous.

Definition

A Pushdown automaton is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$

- Q is set of states
- Σ is the input alphabet
- Γ is the stack alphabet
- δ is the transition function

$$\delta: Q \times \Sigma \times \Gamma \rightarrow P(Q \times \Gamma)$$

$$\Sigma = \Sigma \cup \epsilon \quad \Gamma = \Gamma \cup \epsilon$$

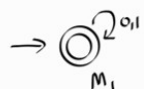
- q_0 is start state

- $F \subseteq Q$ is set of accept states

Decidable problems related to regular languages.

we choose to represent various computational problems by languages.

$$A_{DFA} = \{ \langle B, w \rangle \mid B \text{ is a DFA that accepts input string } w \}$$

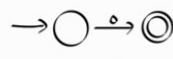


$$\langle M_1, "01" \rangle \in A_{DFA}$$

$$\langle M_1, "000001" \rangle \in A_{DFA}$$

$$\langle M_2, "10" \rangle \in A_{DFA}$$

$$\langle M_2, "001" \rangle \notin A_{DFA}$$



$$L(M_2) = \{0^*1\}$$

Lemma 2.2.1 Proof idea:

We need a way to map derivations of strings using CFG to PDA.

Idea1:

One idea would be to keep the intermediate strings that we produce in the stack.

Why?

PDA needs to find the variables in the intermediate string to make substitutions.

Idea2:

Keep only a part of intermediate string.

Keep any symbols starting with the first variable in the intermediate string.

$$S \rightarrow aSbCTa$$

S
b
c
T
a

Any terminal symbols appearing before the first variable are matched immediately with symbols in the input string.

Here is the general idea for converting the CFG to PDA.

1. Place the marker symbol $\#$ and the start variable on the stack.

2. Repeat the following steps:

a) If the top of the stack is a variable symbol A , nondeterministically select one of the rules for A and substitute A by the string on the right handside of the rule.

b) If the top of the stack is a terminal symbol a , read the next symbol from the input and compare it to a . If they match, repeat. If they do not match, reject on this branch of the nondeterminism.

c) If the top of the stack is symbol $\#$, enter the accept state. Doing so accepts the string if all of it has been read.

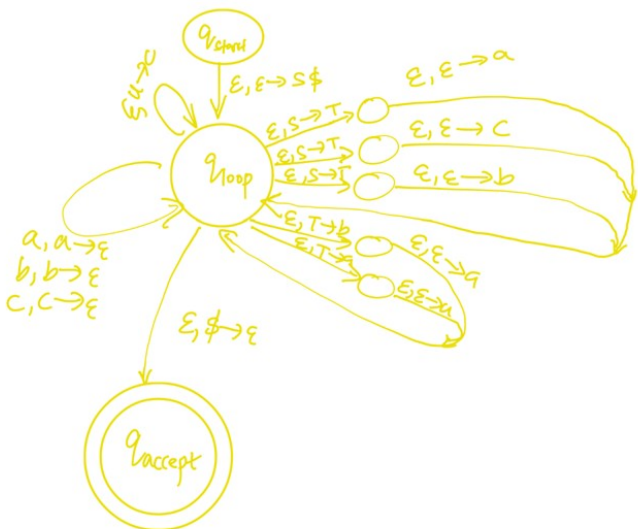
Decidable problems on DFAs

$$A_{DFA} = \{ \langle B, w \rangle \mid B \text{ is a DFA that accepts } w \}$$

$$E_{DFA} = \{ \langle A \rangle \mid A \text{ is a DFA and } L(A) = \emptyset \}$$

$$EQ_{DFA} = \{ \langle A, B \rangle \mid A, B \text{ are DFA and } L(A) = L(B) \}$$

Q1. Convert the following CFG to a PDA. $\Sigma = \{a, b, c\}$

$$S \rightarrow aT \mid cT \mid bT$$
$$T \rightarrow ab \mid Ua$$
$$U \rightarrow c$$


Q2. Show that the language $L = \{a^n b^n c^{2n}\}$ is not a context-free language using the appropriate pumping lemma.

Assume that L is CFL. Then it must follow Pumping lemma. Suppose pumping length is p .

pick $S = a^p b^p c^{2p}$

S should be decomposable into $S = UVXYZ$

1. $\forall x \exists y z \in L$
2. $\forall x y | \leq p$
3. $\forall y | > 0$

- we will consider the V_{xy} region and how we can allocate characters to it.

Case 1: v and y contains two distinct characters. pumping down will change the number of characters and $uv^0xy^0 \notin L$

case 2: v and y contains 'c'. pumping down will create $uv^0xy^0z \notin L$.

case 3: V or Y contains a mix of two characters. pumping up will create a string that has out of order
 $u v^2 x y^2 z \notin L$.

$\therefore L$ does not follow pumping lemma.

L is not a CFL.

Q3. Design CFG for the following language $A = \{ab^nc^{2n} : n \geq 0\}$

$$S \rightarrow aT$$
$$T \rightarrow bTcc \mid \varepsilon$$

Q6. Let $DNA_{DFA} = \{\langle B, w \rangle : B \text{ is a DFA that does not accept } w\}$. Show that DNA_{DFA} is decidable.

Let us create the TM T that decides DNA_{DFA}

$T = \text{"on input } \langle B, w \rangle, \text{ where } B \text{ is a DFA and } w \text{ is a string:}$

1. Simulate B on w .
2. If B ends up at an accept state after we simulate w on B , **reject** otherwise, **accept**.

Simulating B on w will complete in finite amount of time.

$\therefore$ TM T will halt for all inputs.

Q5. Prove that $NOODD_{DFA} = \{ \langle A \rangle : A \text{ is a DFA and } L(A) \text{ does not contain any strings of odd length} \}$ is decidable.

We will create TM K which is a decider for $NOODD_{TM}$.

$K =$ "on input $\langle A \rangle$, where A is a DFA:

1. Construct DFA B, where $L(B) = (\Sigma^*) \cdot (\Sigma)$
2. Construct DFA C, where $L(C) = L(B) \cap L(A)$
3. Run the decider E_{DFA} on $\langle C \rangle$
4. If the decider E_{DFA} accepts $\langle C \rangle$, accept.
If it rejects, reject.

B and C can be constructed in finite amount of time. the decider for E_{DFA} is guaranteed to halt.

∴ K is guaranteed to halt.